TYPES OF BOOSTING ALGORITHMS

ADABOOST|XGBOOST|LIGHTGBM

1.ADABOOST(Adaptive Boosting)

DEFINITION:

Adaptive Boosting Combines many weak decision trees to make one strong predictor.

It gives more weight to mistakes so the next tree learns from the errors.

HOW IT WORKS:

Step 1: Train weak model

Step 2: Find wrong predictions

Step 3: Give more weight to wrong ones

Step 4: Repeat and combine

PROS AND CONS

PROS:

1. Simple to understand and implement
2. Less prone to overfitting
3. Automatically adjusts weights for errors

CONS:

1. Sensitive to noisy data
2. Slow training
3. Needs good quality data

DIAGRAM

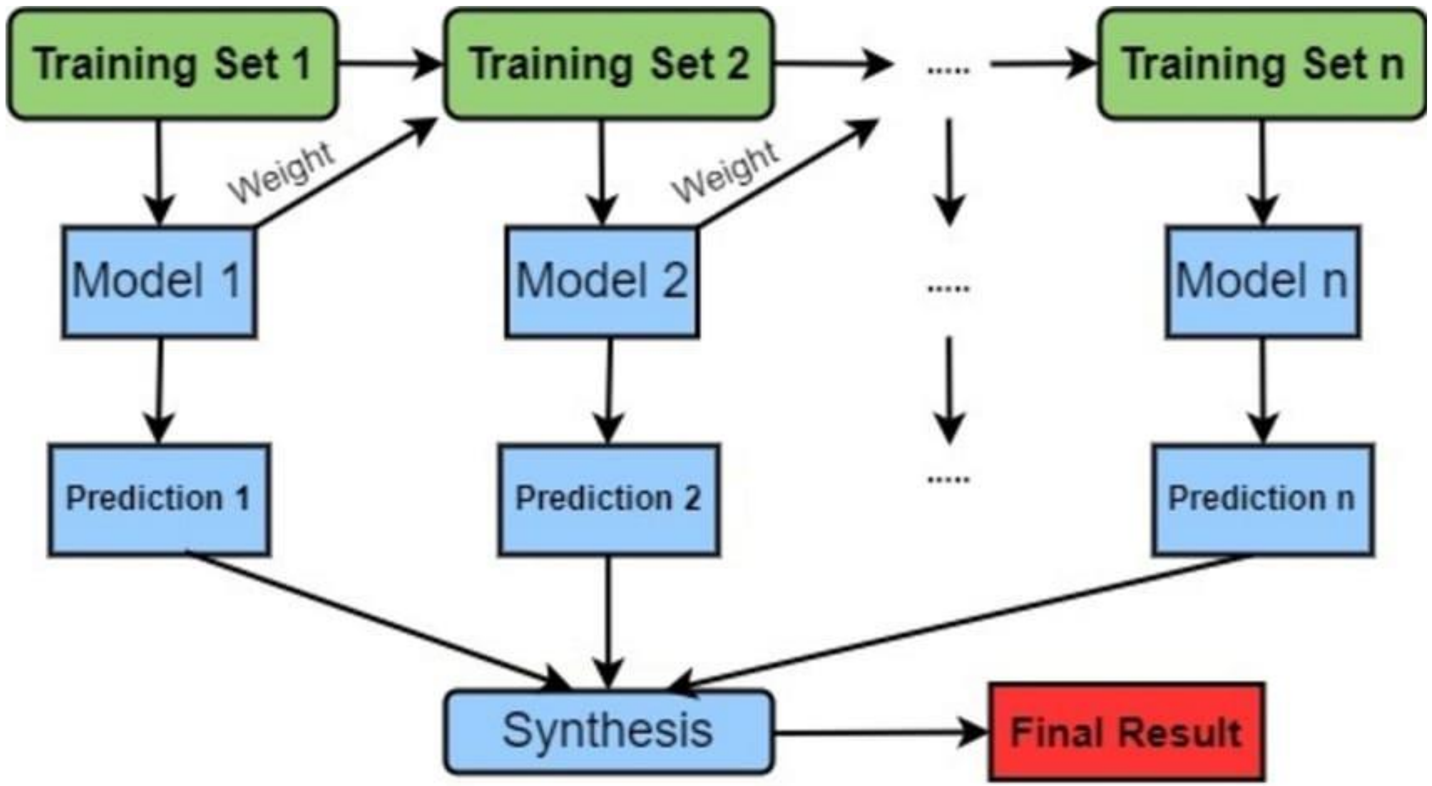

Training Set 1
Training Set 2
.....
Training Set n
Weight
Weight
Model 1
Model 2
.....
Model n
Prediction 1
Prediction 2
.....
Prediction n
Synthesis
Final Result

DIAGRAM EXPLANATION

AdaBoost works like this:

- Models 1 learns from Training Set 1.
- If Model 1 makes mistakes, it gives more weight to those mistakes.
- Training Set 2 will focus more on those weighted mistakes.
- Model 2 learns, again gives weight to mistakes.
- This continues till Model n.
- Finally, all predictions are combined in Synthesis to get Final Result.

2.XGBOOST(Extreme Gradient Boosting)

DEFINITION:

An advanced, highly optimized version of Gradient Boosting. It builds trees level-wise and uses gradients to minimize errors with regularization.

HOW IT WORKS:

Step 1: Calculate residual errors

Step 2: Build new tree to fix errors

Step 3: Use Gradient Descent to minimize loss

Step 4: Combine all trees

PROS AND CONS

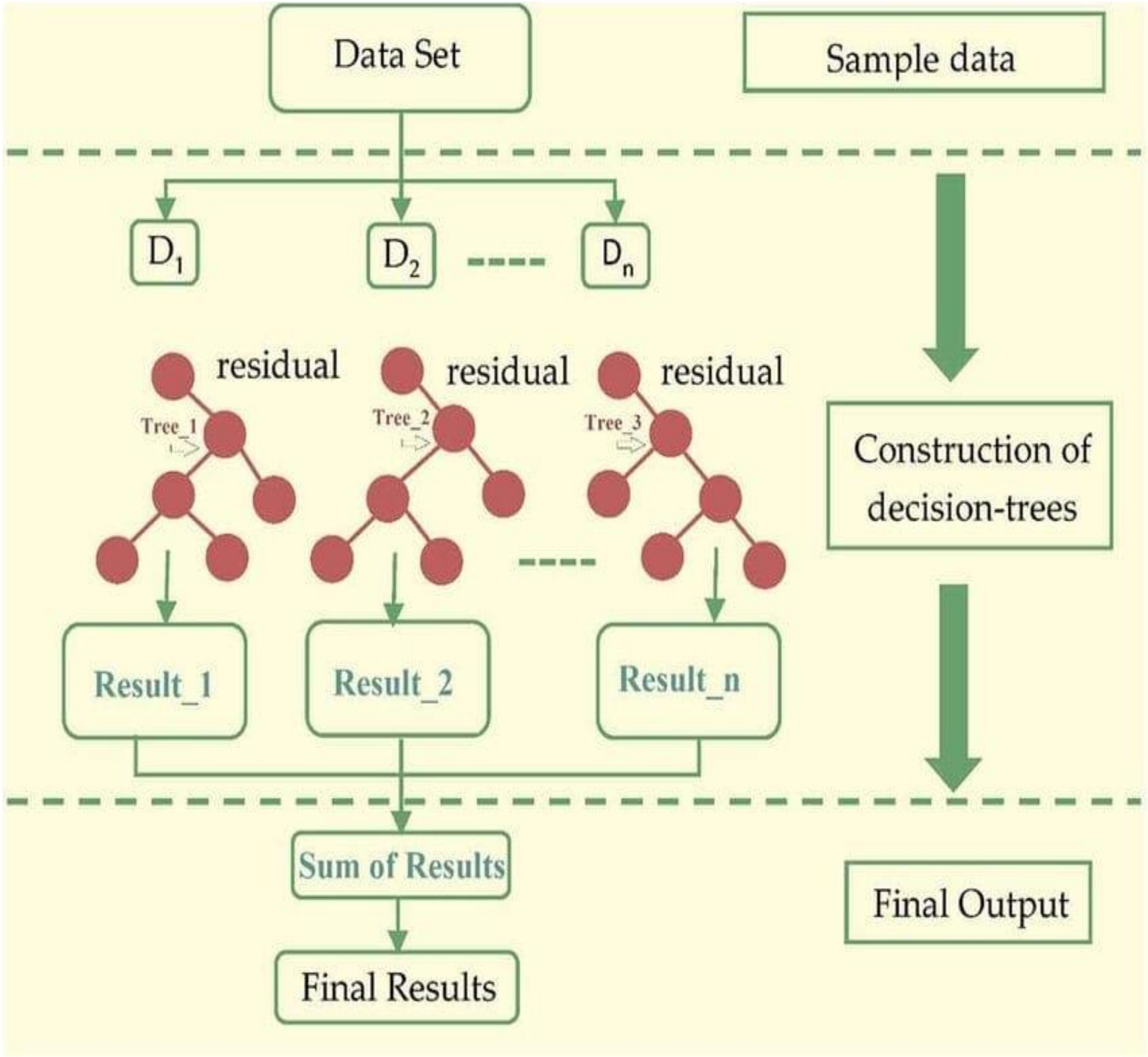
PROS:

1. Very high accuracy
2. Handles missing values automatically
3. Winner of many Kaggle competitions

Cons:

1. Slow and needs more memory
2. Many parameters to tune
3. Level-wise growth is slower

DIAGRAM



Data Set
Sample data
D_1
D_2
D_n
residual
residual
residual
Tree_1
Tree_2
Tree_3
Construction of decision-trees
Result_1
Result_2
Result_n
Sum of Results
Final Results
Final Output

DIAGRAM EXPLANATION

XGBoost works like this:

- Models 1 learns from Data Set and gives Initial Prediction.
- If Model 1 makes mistakes, it calculates the residual error.
- Tree 2 will learn that residual to correct the error.
- This continues till Tree n.
- Finally, all results are summed to get Final Result.

3.LGBOOST(Light Gradient Boosting)

DEFINITION:

Light Gradient Boosting Machine, developed by Microsoft. It grows trees leaf-wise (only where loss is maximum), making it 10x faster and more accurate.

HOW IT WORKS:

Step 1: Find best leaf to split (not level)

Step 2: Grow leaf-wise, not level-wise

Step 3: Very fast with less memory

Step 4: Best accuracy

PROS AND CONS

PROS:

1. 10x Faster than XGBoost
2. Low memory usage
3. Highest accuracy (R2: 0.89137)
4. Leaf-wise growth - more efficient

CONS:

1. Can overfit on small datasets
2. Needs more data to perform well

DIAGRAM

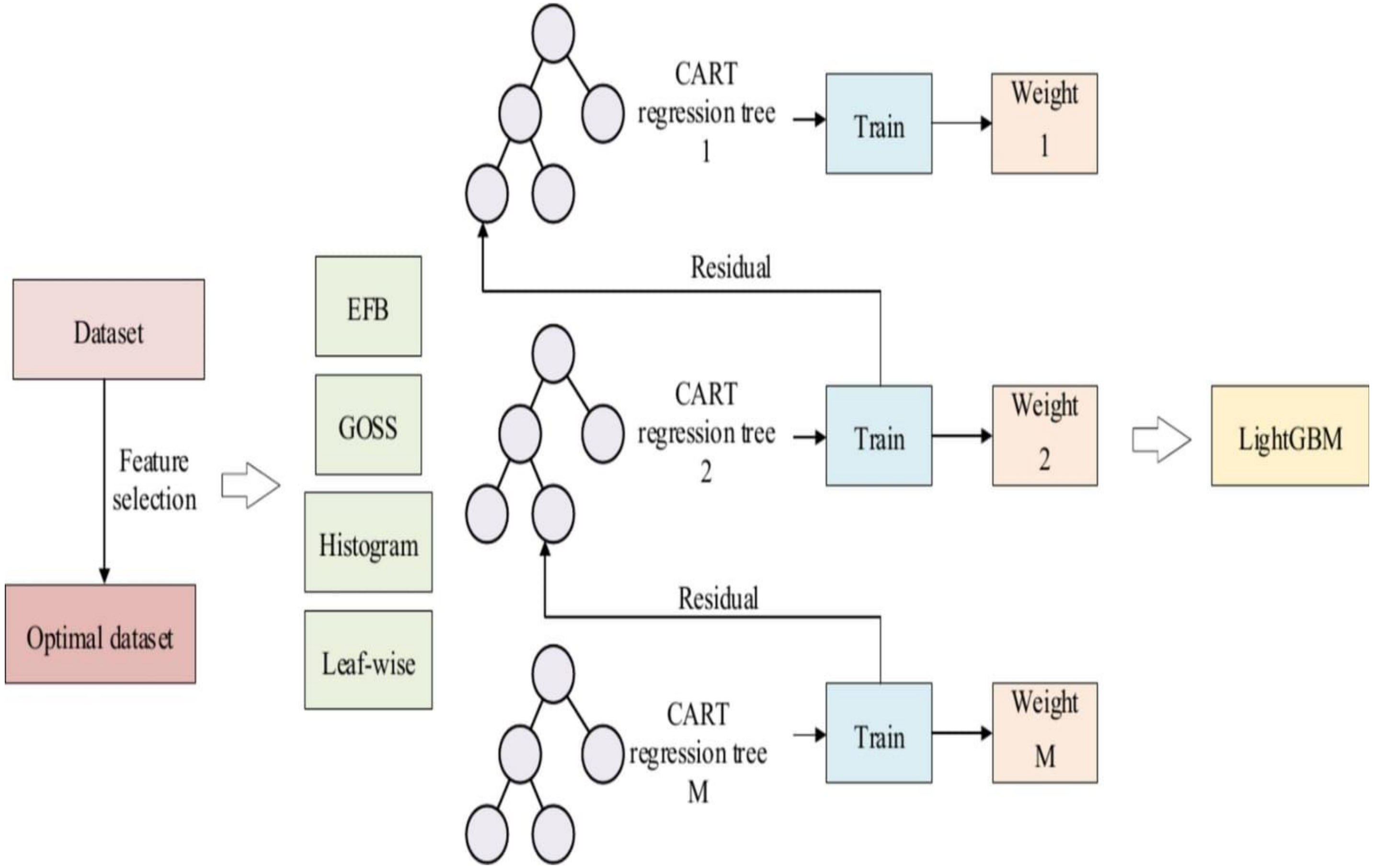

Dataset
Feature selection
Optimal dataset
EFB
GOSS
Histogram
Leaf-wise
CART regression tree 1
Train
Weight 1
Residual
CART regression tree 2
Train
Weight 2
LightGBM
Residual
CART regression tree M
Train
Weight M

DIAGRAM EXPLANATION

LGBoost works like this:

- Dataset to Optimal Dataset: First, the raw dataset undergoes feature selection to get an optimal dataset.
- 4 Main Techniques: LightGBM uses 4 key techniques to become fast and efficient - EFB (Exclusive Feature Bundling), GOSS (Gradient-based One-Side Sampling), Histogram, and Leaf-wise growth.
- Boosting Process: It trains up to M CART regression trees sequentially. The error or Residual from one tree is passed to the next tree to improve accuracy.
- Final Model: All the individual weights from each tree are combined together to form the final LightGBM model.

CONCLUSION

- There are various types of boosting algorithms like AdaBoost, Gradient Boosting, XGBoost, and LightGBM
- Each algorithm has its own advantages. While AdaBoost is simple and good for beginners, XGBoost and LightGBM provide higher accuracy and speed for large datasets.
- In summary, the choice of boosting algorithm depends on the dataset size, accuracy requirement, and computational efficiency. Among them, LightGBM and XGBoost are currently the most powerful and widely used.